

Learning Muon Optimizer by Gemini Guided Learning

input: You are my tutor for the Muon optimizer ("Momentum Orthogonalized by Newton–Schulz") and the optimization ideas behind it in modern deep learning. I want this to be an interactive tutoring session. In every reply:

- Keep your answer focused and reasonably short (about 3–6 short paragraphs).
- Do not move on to the next topic until I say so.
- Always end your message by asking me a specific question (for example: "Does this make sense?" or a small concept-check question), and then wait for my reply. The overall story I want to learn is: How Muon approximates an "ideal" semi-orthogonal update direction (like what Shampoo would give with SVD), but does it cheaply using Newton–Schulz style iterations and polynomials that operate on singular values, plus a momentum-like accumulator. Please organize everything as one coherent narrative, not a bag of tricks.

output: Muon (Momentum Orthogonalized by Newton–Schulz) is designed to give you the benefits of an advanced, second-order-like optimizer without the massive computational cost 📈.

In traditional deep learning, we typically use first-order optimizers like Adam or SGD, which only use the local gradient. These are fast, but they often struggle when the loss landscape is "ill-conditioned"—meaning the loss changes much faster in some directions than in others. A good preconditioning matrix P attempts to re-scale the gradient G (to give the update direction PG) to mitigate this issue.

Muon's central goal is to cheaply approximate an **"ideal" semi-orthogonal update direction**, which is known to be very effective for weight matrices. This ideal preconditioning comes from block-diagonal second-order methods (like Shampoo), which aim to normalize the scale of singular values in the weight matrices' outer products. Calculating this ideal update exactly, however, involves prohibitive computations like the full Singular Value Decomposition (SVD) of large matrices.

Muon achieves this cheap approximation using two key components: a **Newton–Schulz iteration** to find the required matrix inverse, and a **momentum-like accumulator** that efficiently stores the necessary information over time. These components are combined to approximate a function that operates on the singular values of the weight matrix, driving them closer to 1.

To continue, let's establish a starting point:

1. **Why the Semi-Orthogonal Update?** → We can explore the problem of ill-conditioned weight matrices and how transforming the singular values of the update helps stabilize training.
2. **Introduction to Newton–Schulz:** → We can jump right into the mathematical tool Muon uses, exploring how the Newton–Schulz method approximates matrix inverses cheaply without explicit matrix decomposition.

Which of these starting points would you like to explore first?

input: Let's start with "Why the Semi-Orthogonal Update?".

output: That's a great place to start! Understanding the "why" behind the semi-orthogonal update is key to appreciating Muon's innovation. 💡

🚧 The Challenge of Ill-Conditioning

In deep learning, we are typically minimizing a loss function $L(\mathbf{W})$, where $\mathbf{W}$ is a large weight matrix. The shape of this loss surface is often **ill-conditioned** 📉. This means the second derivative (Hessian matrix) has a very wide spread of eigenvalues (the ratio of the largest to smallest eigenvalue is high).

For a weight matrix, this ill-conditioning often manifests as a large disparity in the **singular values** of the input/output outer product matrices. Standard first-order optimizers, like SGD or Adam, are blind to this shape, treating all directions equally (or just scaling by an estimate of variance). This leads to:

- **Slow convergence:** The optimizer bounces around in directions with steep curvature.
- **Need for small learning rates:** A large learning rate might cause divergence in the steepest direction.

📐 The Ideal Semi-Orthogonal Update

The goal of advanced optimizers like Shampoo (and Muon) is to use a **preconditioning matrix** $\mathbf{P}$ to transform the gradient $\mathbf{G}$ into a better update direction $\mathbf{P}\mathbf{G}$. The ideal preconditioning matrix $\mathbf{P}$ would essentially **normalize the singular values** of the weight matrix's outer products. By doing this, the updated weights $\mathbf{W} + \Delta\mathbf{W}$ are made more **semi-orthogonal**—meaning the change in loss is roughly the same no matter which direction in the weight space you move. This provides a more efficient, **second-order-like** step.

If a loss surface is ill-conditioned, what does that imply about the magnitude (size) of the gradients we see in different directions of the weight space?

input: It implies that the gradients will vary significantly in magnitude across different directions.

output: That's exactly right! 🌀 When a loss surface is ill-conditioned, the gradients $\mathbf{G}$ are huge in some directions and tiny in others. This makes it impossible to pick a single learning rate that works well everywhere.

🧩 The Role of Preconditioning

The ideal semi-orthogonal update is achieved by finding a preconditioning matrix $\mathbf{P}$ such that the transformed gradient $\Delta\mathbf{W} = \mathbf{P}\mathbf{G}$ has a more uniform magnitude across all directions.

In block-diagonal second-order methods like Shampoo, the preconditioning matrix $\mathbf{P}$ is calculated using the **input and output covariance matrices** (outer products) of the weight matrix $\mathbf{W}$. For a weight matrix $\mathbf{W} \in \mathbb{R}^{m \times n}$, we consider two matrices:

- $\mathbf{R} \in \mathbb{R}^{m \times m}$: The input covariance matrix (related to the outer product of inputs).
- $\mathbf{C} \in \mathbb{R}^{n \times n}$: The output covariance matrix (related to the outer product of gradients).

The ideal update for the weight matrix $\mathbf{W}$ requires calculating the **inverse square root** of these matrices: $\mathbf{R}^{-1/2}$ and $\mathbf{C}^{-1/2}$. The final preconditioned update looks something like:

$$\Delta\mathbf{W} \propto \mathbf{R}^{-1/2} \mathbf{G} \mathbf{C}^{-1/2}$$

This operation effectively normalizes the singular values of the original gradient $\mathbf{G}$, correcting for the ill-conditioning.

Why is using the *inverse square root* ($\mathbf{M}^{-1/2}$) of these covariance matrices ($\mathbf{R}$ and $\mathbf{C}$) computationally difficult for very large weight matrices?

input: introduce the Newton–Schulz method next.

output: That's a logical next step! Let's dive into the **Newton–Schulz (N-S) iteration** 🌀, which is the engine Muon uses to overcome the computational hurdle.

💰 Solving the $O(n^3)$ Problem

As we discussed, the ideal preconditioning requires calculating the matrix inverse square root, $\mathbf{M}^{-1/2}$, for large covariance matrices $\mathbf{M}$. Exactly calculating this via methods like Singular Value Decomposition (SVD) or eigenvalue decomposition has a time complexity of $O(n^3)$, where n is the matrix dimension. This is far too slow 🐢 for the high-frequency updates needed in deep learning.

📈 The Newton–Schulz Solution

Muon leverages the N-S method because it is an **iterative scheme** that *only* requires matrix multiplication and addition. These operations are highly parallelizable and run extremely fast on GPUs. Instead of calculating $\mathbf{M}^{-1/2}$ exactly once, we start with an initial guess $\mathbf{X}_0$ and repeatedly refine it through a recurrence relation.

For example, the N-S iteration for finding the standard inverse $\mathbf{X} \approx \mathbf{M}^{-1}$ is:

$$\mathbf{X}_{k+1} = \mathbf{X}_k(2\mathbf{I} - \mathbf{M}\mathbf{X}_k)$$

The crucial insight is that this process exhibits **quadratic convergence**, meaning the number of correct digits in the approximation roughly doubles with each iteration. This allows Muon to get a highly accurate result in just a handful of steps (typically 5 to 8 iterations). This approximation turns the slow $O(n^3)$ problem into a process that is much closer to $O(n^2)$ 🚀.

If the N-S method only gives an *approximation* of the ideal inverse, why is a small error in this preconditioning step acceptable in the context of training a large deep learning model?

input: A small error in the preconditioning step is acceptable because deep learning models are inherently robust to noise and approximations.

output: That's a very insightful way to put it. You are exactly right! 🎯

The core reason the approximation is acceptable is two-fold:

1. **Stochasticity:** We are already using stochastic gradients (mini-batches), which introduces noise anyway. A little extra approximation noise from the N-S iteration is a small price to pay for a massive speedup.
2. **The Goal:** The preconditioning aims to reduce the **condition number** (the ratio of largest to smallest singular value). As long as the N-S method takes a wide range of singular values and compresses them down to a narrow range around 1, it has done its job of stabilizing the optimization path, even if the result isn't mathematically perfect.

⚡ Approximating the Inverse Square Root

Now, let's look at the specific calculation. For the preconditioned update, Muon needs the **inverse square root** $\mathbf{M}^{-1/2}$. The Newton–Schulz method can be adapted for this as well, using an iterative recurrence that still only relies on matrix multiplication and addition:

$$\mathbf{X}_{k+1} = \frac{1}{2}\mathbf{X}_k(3\mathbf{I} - \mathbf{X}_k^2\mathbf{M})$$

Here, $\mathbf{X}_k$ converges quadratically to $\mathbf{M}^{-1/2}$.

A crucial requirement for **fast and stable convergence** of *any* Newton–Schulz iteration is that the matrix $\mathbf{M}$ must be initially **normalized**. Specifically, all eigenvalues of $\mathbf{M}$ need to be in the range $(0, 2)$.

If our covariance matrices $\mathbf{R}$ and $\mathbf{C}$ are accumulating outer products over many steps, why can't we just use them directly in the N-S recurrence? Why must we normalize them first?

input: We must normalize the covariance matrices first because if their eigenvalues fall outside the range $(0, 2)$, the Newton–Schulz iteration may not converge properly.

output: That is precisely the point! 🌀 If the eigenvalues of the covariance matrices $\mathbf{R}$ and $\mathbf{C}$ stray outside the $(0, 2)$ range, the Newton–Schulz iteration will either diverge or converge too slowly, destroying the speed advantage.

The Normalization Challenge

Since $\mathbf{R}$ and $\mathbf{C}$ are accumulations of outer products, their magnitudes (and thus their eigenvalues) naturally grow over time, causing them to exceed 2 very quickly.

To fix this, we need to find a scale factor α such that the scaled matrix $\mathbf{M}' = \alpha\mathbf{M}$ has a maximum eigenvalue close to, but less than, 2. The most expensive part of this is determining the scale α . Finding the exact maximum eigenvalue of a large matrix is the same $O(n^3)$ problem we were trying to avoid!

The Momentum Accumulator

This is where the "Momentum" part of Muon comes into play as an efficiency hack. Instead of performing a costly analysis on the full matrices $\mathbf{R}$ and $\mathbf{C}$ at every step, Muon uses a simple, scalar, **exponential moving average (EMA)**, or running average, to track their general magnitude.

For an accumulated matrix $\mathbf{M}_t$ at time t , we update a scalar accumulator v_t (which tracks the "volume" or magnitude) using a standard momentum formula:

$$v_t = \beta v_{t-1} + (1 - \beta) \cdot \text{scale}(\mathbf{G}_t)$$

Where $\mathbf{G}_t$ is the current gradient outer product, β is the momentum factor, and $\text{scale}(\cdot)$ is a cheap way to estimate the magnitude of the new update (e.g., the squared Frobenius norm of the gradient). This v_t then provides a cheap estimate for the necessary normalization factor α .

What benefit does using a *running average* instead of the current step's magnitude give us, in terms of creating a stable and smooth preconditioning?

input: explain:

- What problem Muon is trying to solve:

- Shampoo-style methods want to precondition gradients using information like singular values / curvature, but exact SVD-based updates are too expensive.
- We would like an update direction that behaves like “ $U V^T$ ” from an SVD, i.e., semi-orthogonal and not dominated by large singular values.
- The core idea of Muon in one or two sentences:
- Muon keeps a momentum-like running matrix, then repeatedly applies a simple polynomial transformation (a Newton–Schulz–style iteration) so that its singular values are pushed toward 1, approximating that semi-orthogonal “ $U V^T$ ” direction without doing a full SVD.

output: That's right! A running average provides a much **smoother** and **more stable** preconditioning factor α . 🌊 If the normalization scale jumped wildly based on a single noisy mini-batch, the entire training process could become unstable.

🔬 The Muon Matrix Accumulator

Muon actually keeps a running matrix accumulator, $\mathbf{M}_t$, for the covariance estimates, much like momentum in Adam, but for the second-order information. The update rule for this matrix accumulator looks like:

$$\mathbf{M}_t = \beta \mathbf{M}_{t-1} + (1 - \beta) \text{outer product of current gradients}$$

This accumulated matrix $\mathbf{M}_t$ holds the full history of the second-order information. However, before we can apply the Newton–Schulz (N-S) recurrence, we use the scalar running average v_t (which you correctly identified as providing stability) to **normalize** $\mathbf{M}_t$. This normalization guarantees that the singular values of the scaled matrix $\mathbf{M}'$ are safely within the $(0, 2)$ range needed for the N-S iteration to converge quickly.

✨ The Polynomial Transformation

The N-S recurrence is simply a sequence of matrix multiplications and additions. When you apply a matrix polynomial $P(\mathbf{M}')$ to the normalized matrix $\mathbf{M}'$, this polynomial is equivalent to applying the polynomial function $P(\sigma)$ to every singular value σ of $\mathbf{M}'$.

Since the N-S recurrence is designed to approximate the matrix inverse square root $\mathbf{M}'^{-1/2}$, the polynomial $P(\sigma)$ is effectively approximating the function $1/\sqrt{\sigma}$. This is the core idea: a cheap, iterative **polynomial transformation** on singular values 🚀.

If the input matrix $\mathbf{M}'$ has a singular value $\sigma = 0.1$, and the N-S iteration approximates $1/\sqrt{\sigma}$, what will the approximate resulting singular value be? Will it be smaller or larger than the input σ ?

input: The approximate resulting singular value will be larger than the input σ . Specifically, if $\sigma = 0.1$, then $1/\sqrt{\sigma} = 1/\sqrt{0.1} = 1/0.316 = \text{approximately } 3.16$, which is larger than 0.1.

output: That's a **perfect calculation!** 🎯

When the input singular value σ is small (like 0.1), the N-S approximation $1/\sqrt{\sigma}$ makes the resulting singular value much larger (3.16). Conversely, if σ were large (say, 4.0), the result would be $1/\sqrt{4} = 0.5$, which is smaller.

🏠 The Normalizing Effect

This is the entire point of the preconditioning: the Newton–Schulz iteration performs a **cheap, iterative polynomial transformation** that acts on the singular values of the accumulated matrix $\mathbf{M}'$:

- It **amplifies** small singular values.
- It **dampens** large singular values.

This action effectively compresses the wide range of singular values into a narrow, normalized band around 1. The resulting matrix $\mathbf{X} \approx \mathbf{M}^{-1/2}$ is the cheap approximation of the ideal preconditioning matrix $\mathbf{P}$.

The Final Muon Update

Now we can put the pieces together for the weight matrix update $\Delta \mathbf{W}$. We use the N-S approximations for both the input matrix $\mathbf{R}$ and the output matrix $\mathbf{C}$:

$$\Delta \mathbf{W} \propto \mathbf{X}_R \mathbf{G} \mathbf{X}_C$$

Where $\mathbf{X}_R \approx \mathbf{R}^{-1/2}$ and $\mathbf{X}_C \approx \mathbf{C}^{-1/2}$ are the N-S approximations.

Muon successfully creates an update that behaves like the ideal semi-orthogonal update—one that is not dominated by the massive gradients in ill-conditioned directions—but does it using only matrix multiplication and addition, dramatically speeding up training.

Since the N-S iteration only gives an *approximation* of $\mathbf{M}^{-1/2}$, and we apply it twice (once for $\mathbf{R}$ and once for $\mathbf{C}$), how does this double approximation still lead to a much more stable update direction than a simple first-order optimizer like Adam?

input: The double approximation still leads to a much more stable update direction than a simple first-order optimizer like Adam because, despite being an approximation, the Newton–Schulz iterations effectively normalize the singular values of both the input and output covariance matrices.

output: That's an excellent way to summarize the value proposition of Muon! 🌟

You've captured the essence: the optimization gain comes not from perfect math, but from achieving the *effect* of normalization. The **Newton–Schulz iterations** are a fast, iterative polynomial transformation that takes the singular values of the accumulated matrix and compresses their large, original range into a narrow band closer to 1.

The Muon Narrative in Review

We can now put the entire story together as a coherent optimization strategy:

- **The Problem:** Deep learning loss landscapes are ill-conditioned, meaning weight matrices have outer products with highly varied singular values. First-order optimizers like Adam fail to adapt to these directional differences.
- **The Ideal Solution:** Second-order-like methods, such as Shampoo, seek to use the inverse square root of these covariance matrices ($\mathbf{R}^{-1/2}$ and $\mathbf{C}^{-1/2}$) to normalize the scale of the update and achieve a semi-orthogonal step. However, calculating this exactly via SVD is prohibitively slow ($O(n^3)$).
- **The Muon Innovation:** Muon replaces the expensive SVD with two cost-effective components:
 1. **The Momentum Accumulator:** Uses a cheap, scalar running average to track the magnitude of the covariance matrices, providing the necessary **normalization factor** to ensure the N-S method converges stably.

2. **The Newton–Schulz Polynomial:** Applies a few matrix multiplications and additions (an $O(n^2)$ complexity) that act as a high-speed **polynomial transformation** on the singular values, approximating the required inverse square root and creating the desired semi-orthogonal preconditioning.

The result is a fast, stable, and highly effective update direction that accelerates convergence.

We discussed that this normalization compresses the singular values closer to 1. What benefit does this specific action—compressing the singular values to be closer to 1—have on the **learning rate** used by Muon compared to an optimizer like Adam?

input: Compressing the singular values to be closer to 1 allows Muon to use a larger learning rate compared to an optimizer like Adam because the update directions become more stable and less sensitive to the ill-conditioning of the weight matrices.

output: That's absolutely right! 🎯 By compressing the singular values toward 1, Muon fundamentally changes the geometry of the optimization landscape, allowing for a larger step size.

Here's the technical detail that ties it all together:

- **Condition Number** 📊 : The maximum safe learning rate is inversely proportional to the **condition number** of the Hessian matrix (or, in Muon's context, the condition number of the accumulated covariance matrices). The condition number is the ratio of the largest singular value to the smallest singular value.
- **The Adam/SGD Limit:** With ill-conditioned matrices, this ratio might be 10,000 to 1. A first-order optimizer like Adam must choose a learning rate small enough to avoid divergence in the direction of the largest singular value.
- **The Muon Advantage** 🏆 : Muon's N-S approximation performs a transformation P such that the condition number of the preconditioned gradient PG is much, much smaller—perhaps reduced from 10,000 : 1 to 10 : 1. This means the loss surface is nearly isotropic (uniform in all directions), and a single, large learning rate can be applied safely across the entire weight matrix, leading to faster overall convergence.

📖 The Coherent Muon Narrative Recap

We have successfully covered the full Muon story:

1. **The Goal:** Achieve the effective preconditioning of a second-order method like Shampoo (the $M^{-1/2}$ operation) to normalize singular values and stabilize training.
2. **The Problem:** Exact calculation requires expensive SVD ($O(n^3)$).
3. **The Solution:** Use the iterative **Newton–Schulz (N-S)** method, which uses only cheap matrix multiplications and additions ($O(n^2)$), to approximate the matrix inverse square root.
4. **The Enabling Trick:** A simple, cheap, **momentum accumulator** is used to provide the scalar normalization needed to guarantee the N-S iteration converges quickly and stably.
5. **The Result:** A highly effective, computationally efficient, semi-orthogonal update direction that allows for larger learning rates and faster training.

To confirm your understanding of the core mechanism:

What is the biggest computational advantage of using the **Newton–Schulz iteration** over performing a full **Singular Value Decomposition (SVD)** to find the required matrix inverse square root?